

Low-rank Connectivity and the Dynamics It Produces

A short note on the framework used by Dubreuil et al. (2022)

1 The starting point: a recurrent neural network

The paper studies firing-rate recurrent neural networks. A common continuous-time form is

$$\tau \frac{dx_i}{dt} = -x_i + \sum_{j=1}^N J_{ij} \phi(x_j) + \sum_{s=1}^{N_{\text{in}}} I_i^{(s)} u_s(t), \quad (1)$$

where:

- $x_i(t)$ is the activation of neuron i ;
- $\phi(x_i)$ is its firing rate after a static nonlinearity;
- J_{ij} is the recurrent weight from neuron j to neuron i ;
- $u_s(t)$ are external inputs, such as stimulus or context signals;
- $I_i^{(s)}$ is the input weight from input channel s to neuron i .

In vector form,

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + J\phi(\mathbf{x}) + \sum_s \mathbf{I}^{(s)} u_s(t). \quad (2)$$

A completely general $N \times N$ recurrent matrix J can have rank as large as N . Such a matrix can create many independent recurrent feedback directions. The low-rank assumption sharply restricts this.

2 What “low rank” means

A rank- R recurrent connectivity matrix can be written as a sum of R outer products:

$$J = \frac{1}{N} \sum_{r=1}^R \mathbf{m}^{(r)} \left(\mathbf{n}^{(r)} \right)^\top, \quad (3)$$

or componentwise,

$$J_{ij} = \frac{1}{N} \sum_{r=1}^R m_i^{(r)} n_j^{(r)}. \quad (4)$$

Each term has two population-length vectors:

- $\mathbf{n}^{(r)}$ is a *read-in* or *selection* vector. It defines which population activity pattern is measured.
- $\mathbf{m}^{(r)}$ is a *write-out* vector. It defines which pattern of recurrent input is sent back into the neurons.

For rank one,

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top, \quad (5)$$

so recurrent processing has a simple form:

$$J\phi(\mathbf{x}) = \mathbf{m} \left(\frac{1}{N} \mathbf{n}^\top \phi(\mathbf{x}) \right). \quad (6)$$

The network first projects the current firing rates onto $\mathbf{n}$, producing one scalar, and then feeds that scalar back along $\mathbf{m}$.

3 The latent variable

The scalar produced by the projection is the key low-dimensional variable:

$$\kappa(t) = \frac{1}{N} \sum_{j=1}^N n_j \phi(x_j(t)). \quad (7)$$

For rank one, the recurrent input to neuron i is simply

$$m_i \kappa(t). \quad (8)$$

Thus the full N -dimensional network is not using an arbitrary recurrent input pattern. Its recurrent input always lies along the single population direction $\mathbf{m}$. For rank R , there are R such latent variables:

$$\kappa_r(t) = \frac{1}{N} \sum_{j=1}^N n_j^{(r)} \phi(x_j(t)), \quad r = 1, \dots, R. \quad (9)$$

The recurrent input is then

$$J\phi(\mathbf{x}) = \sum_{r=1}^R \mathbf{m}^{(r)} \kappa_r(t). \quad (10)$$

This is the main mathematical reason low-rank networks are interpretable: the large neural population can be described through a small set of recurrent latent variables $\kappa_1, \dots, \kappa_R$.

4 Why low-rank connectivity creates low-dimensional dynamics

Insert the low-rank form into the RNN:

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \sum_{r=1}^R \mathbf{m}^{(r)} \kappa_r(t) + \sum_s \mathbf{I}^{(s)} u_s(t). \quad (11)$$

This equation says that changes in the neural state are built only from:

- the recurrent directions $\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(R)}$;
- the input directions $\mathbf{I}^{(1)}, \dots, \mathbf{I}^{(N_{\text{in}})}$;
- the leak term $-\mathbf{x}$.

If the initial state is in the subspace spanned by the recurrent and input directions, it stays there. Even when the full network has thousands of neurons, the meaningful trajectories can live in a much smaller subspace. This is what the paper means by reducing the network to latent dynamics.

5 A rank-one example

With one input $u(t)$ and rank-one recurrent connectivity,

$$\tau \dot{x}_i = -x_i + m_i \kappa(t) + I_i u(t), \quad (12)$$

$$\kappa(t) = \frac{1}{N} \sum_j n_j \phi(x_j(t)). \quad (13)$$

If ϕ were linear, the latent dynamics would be approximately

$$\tau \dot{\kappa} = -\kappa + \sigma_{nm} \kappa + \sigma_{nI} u(t), \quad (14)$$

where

$$\sigma_{nm} = \frac{1}{N} \sum_i n_i m_i, \quad \sigma_{nI} = \frac{1}{N} \sum_i n_i I_i. \quad (15)$$

These overlaps control the computation:

- σ_{nI} says how strongly the input drives the latent variable;
- σ_{nm} says how strongly the latent variable feeds back onto itself.

If feedback is weak, κ tracks the input briefly. If feedback is tuned close to the leak, κ can integrate information over time. If the nonlinearity is strong and feedback is positive, the same low-dimensional system can support attractor-like decision states.

6 Where population structure enters

The paper then asks: what statistics of the neuron-specific parameters are needed for a task? For each neuron, collect its coordinates in connectivity space:

$$\left(m_i^{(1)}, \dots, m_i^{(R)}, n_i^{(1)}, \dots, n_i^{(R)}, I_i^{(1)}, \dots, I_i^{(N_{\text{in}})}, w_i \right). \quad (16)$$

Across neurons, these coordinates form a distribution.

Single random population

In a fully random population structure, these neuron coordinates are modeled as draws from one joint distribution, often a multivariate Gaussian:

$$\mathbf{a}_i \sim \mathcal{N}(0, \Sigma). \quad (17)$$

The effective latent dynamics are governed by global covariances such as σ_{nm} and σ_{nI} . There are no distinct groups of neurons with different computational roles.

Multiple subpopulations

In a non-random population structure, the coordinates are better described as a mixture:

$$p(\mathbf{a}) = \sum_{p=1}^P \alpha_p \mathcal{N}(\mu_p, \Sigma_p). \quad (18)$$

Each component corresponds to a subpopulation. The subpopulations can have different covariances, so the same input may modulate one group strongly and another weakly.

In the nonlinear case, this is crucial. Dubreuil et al. write the effective couplings as population-weighted quantities of the form

$$\tilde{\sigma}_{ab} = \sum_p \alpha_p \sigma_{ab}^{(p)} \langle \phi' \rangle_p. \quad (19)$$

Here $\langle \phi' \rangle_p$ is the gain of subpopulation p . Inputs can change these gains differently across subpopulations. This lets context reshape the effective latent dynamics without necessarily driving the same latent variable directly.

7 The computational message

Low rank controls the *dimension* of the recurrent dynamics. A rank- R network has R main recurrent latent variables.

Population structure controls how inputs, feedback, and readout are arranged across neurons. A single random population can implement many tasks by using low-dimensional dynamics. But tasks requiring flexible input-output remapping can require multiple subpopulations, because different groups need to have different gain modulations and different effective roles.

In one sentence:

Low-rank connectivity says the network computes through a small number of collective variables; non-random population structure says different neurons are organized so that inputs can flexibly reshape the dynamics of those collective variables.

Glossary

Term	Meaning
J	recurrent connectivity matrix
rank R	number of independent recurrent feedback directions
$\mathbf{n}^{(r)}$	vector that reads population activity into latent variable r
$\mathbf{m}^{(r)}$	vector that writes latent variable r back into the population
κ_r	low-dimensional recurrent latent variable
$\mathbf{I}^{(s)}$	input vector for input channel s
$\mathbf{w}$	readout vector for producing the network output
population structure	distribution of neuron coordinates in connectivity space

8 Additional primer for implementing the assignment

The original note above explains why low-rank connectivity is useful. The remaining sections add the implementation details needed to build and train the specific rank-one RNN from the assignment.

9 The concrete model in the task

The task asks for a recurrent neural network with one-dimensional input $u(t) \in \mathbb{R}$, one-dimensional output $z(t) \in \mathbb{R}$, and N recurrent units. The continuous-time state of neuron i is $x_i(t)$. Its firing rate is

$$r_i(t) = \phi(x_i(t)) = \tanh(x_i(t)). \quad (20)$$

The dynamics are

$$\tau \frac{dx_i}{dt} = -x_i + \sum_{j=1}^N J_{ij} \phi(x_j) + I_i u(t), \quad i = 1, \dots, N, \quad (21)$$

with neuronal time constant $\tau = 100$ ms.

The recurrent matrix is rank one:

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top, \quad J_{ij} = \frac{1}{N} m_i n_j. \quad (22)$$

The output is a linear readout of the rates:

$$z(t) = \frac{1}{N} \sum_{i=1}^N w_i \phi(x_i(t)). \quad (23)$$

Only $\mathbf{m}$ and $\mathbf{n}$ are trainable. The input vector $\mathbf{I}$ and output vector $\mathbf{w}$ are fixed after initialization. All trainable and fixed parameters are sampled from standard normal distributions, except $\mathbf{w}$, which is sampled with standard deviation 4.

9.1 Vector form

Let $\mathbf{x}(t) \in \mathbb{R}^N$, $\mathbf{I}, \mathbf{m}, \mathbf{n}, \mathbf{w} \in \mathbb{R}^N$, and $\phi(\mathbf{x})$ mean applying $\tanh$ elementwise. Then

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \frac{1}{N} \mathbf{m} \mathbf{n}^\top \phi(\mathbf{x}) + \mathbf{I} u(t). \quad (24)$$

Because J is rank one, it is better not to explicitly build the full $N \times N$ matrix. Define the scalar recurrent coordinate

$$\kappa(t) = \frac{1}{N} \mathbf{n}^\top \phi(\mathbf{x}(t)). \quad (25)$$

Then the recurrent input is simply

$$J \phi(\mathbf{x}(t)) = \mathbf{m} \kappa(t). \quad (26)$$

So the implemented update can use one dot product with $\mathbf{n}$, then multiply by $\mathbf{m}$. This is both clearer and cheaper than forming J .

10 What an RNN is doing

A recurrent neural network is a dynamical system with memory. A feedforward network computes

$$y = f_\theta(u) \quad (27)$$

from the current input alone. An RNN computes from both the current input and an internal state:

$$\mathbf{x}_{t+1} = F_\theta(\mathbf{x}_t, u_t), \quad z_t = G_\theta(\mathbf{x}_t). \quad (28)$$

The state $\mathbf{x}_t$ carries information from earlier inputs. In this task, the state is a population of firing-rate units. The leak term $-\mathbf{x}$ makes old activity decay, the recurrent term feeds current activity back into the network, and the input term injects the stimulus.

The nonlinearity matters. Here $\phi = \tanh$ maps activations to rates in $(-1, 1)$. Around zero, $\tanh(x) \approx x$, so the network behaves almost linearly. For large positive or negative activations, $\tanh$ saturates and $\phi'(x) = 1 - \tanh^2(x)$ becomes small. Saturation can stabilize activity, but it can also make gradients small during training.

11 From the differential equation to code

The task specifies forward Euler simulation with

$$\Delta t = 20 \text{ ms.}$$

Since $\tau = 100 \text{ ms}$,

$$\alpha = \frac{\Delta t}{\tau} = \frac{20}{100} = 0.2.$$

Forward Euler replaces $\dot{\mathbf{x}}(t)$ by

$$\frac{\mathbf{x}_{t+1} - \mathbf{x}_t}{\Delta t}.$$

Substituting into Equation 24 gives

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha [-\mathbf{x}_t + \mathbf{m}\kappa_t + \mathbf{I}u_t], \quad (29)$$

where

$$\kappa_t = \frac{1}{N} \mathbf{n}^\top \tanh(\mathbf{x}_t). \quad (30)$$

Equivalently,

$$\mathbf{x}_{t+1} = (1 - \alpha)\mathbf{x}_t + \alpha\mathbf{m}\kappa_t + \alpha\mathbf{I}u_t. \quad (31)$$

This is the exact recurrence to implement.

If the stimulus lasts 75 time steps, the real duration is

$$75 \times 20 \text{ ms} = 1500 \text{ ms.}$$

12 Tensor shapes for a batch implementation

Use a batch dimension so many trials can be simulated at once. One convenient layout is:

Quantity	Shape	Meaning
u	(batch, time)	scalar input sequence for each trial
x	(batch, N)	current recurrent state
rates	(batch, N)	$\tanh(\mathbf{x})$
m, n, I, w	(N,)	population vectors
kappa	(batch,)	recurrent latent coordinate
z	(batch,)	scalar output at one time step
outputs	(batch, time)	full output sequence

With this layout, the low-rank computations are:

$$\kappa_t^{(b)} = \frac{1}{N} \sum_{i=1}^N n_i \tanh(x_{t,i}^{(b)}), \quad (32)$$

$$z_t^{(b)} = \frac{1}{N} \sum_{i=1}^N w_i \tanh(x_{t,i}^{(b)}), \quad (33)$$

where b indexes the trial in the batch.

In PyTorch, these become:

```
rates = torch.tanh(x)           # (batch, N)
kappa = (rates * n).mean(dim=1) # (batch,)
z = (rates * w).mean(dim=1)     # (batch,)
```

The use of `mean(dim=1)` implements the factor $1/N$.

13 Initialization and the role of the output scale

The vectors should be initialized as

$$m_i \sim \mathcal{N}(0, 1), \quad n_i \sim \mathcal{N}(0, 1), \quad I_i \sim \mathcal{N}(0, 1), \quad w_i \sim \mathcal{N}(0, 4^2). \quad (34)$$

The larger standard deviation for $\mathbf{w}$ compensates for the averaging in the readout. At random initialization, if the rates are only weakly aligned with $\mathbf{w}$, the sum

$$\frac{1}{N} \sum_i w_i r_i$$

has a typical scale proportional to $\text{std}(w)/\sqrt{N}$ rather than $\text{std}(w)$. A larger output scale helps keep the initial output from being extremely small for moderate N . It does not make $\mathbf{w}$ trainable; it only sets the fixed readout scale.

14 Trainable parameters versus fixed buffers

In PyTorch, trainable parameters should be `nn.Parameter` objects. Fixed tensors that belong to the model should be registered as buffers. Buffers move with the model when calling `model.to(device)`, are saved in the `state_dict`, but are not updated by the optimizer.

For this task:

Vector	PyTorch representation	Optimizer updates it?
m	<code>nn.Parameter</code>	yes
n	<code>nn.Parameter</code>	yes
I	<code>register_buffer("I", ...)</code>	no
w	<code>register_buffer("w", ...)</code>	no

15 A minimal PyTorch module

This is the core implementation pattern. It avoids forming J and unrolls the dynamics over time.

```
import torch
from torch import nn

class RankOneRNN(nn.Module):
    def __init__(self, n_units: int, dt_ms: float = 20.0, tau_ms: float = 100.0):
        super().__init__()
        self.n_units = n_units
        self.alpha = dt_ms / tau_ms

        self.m = nn.Parameter(torch.randn(n_units))
        self.n = nn.Parameter(torch.randn(n_units))

        self.register_buffer("I", torch.randn(n_units))
        self.register_buffer("w", 4.0 * torch.randn(n_units))

    def forward(self, u: torch.Tensor, x0: torch.Tensor | None = None):
        # u has shape (batch, time)
        batch_size, n_steps = u.shape

        if x0 is None:
            x = u.new_zeros(batch_size, self.n_units)
        else:
            x = x0

        outputs = []
        states = []

        for t in range(n_steps):
            rates = torch.tanh(x)

            z = (rates * self.w).mean(dim=1)
            outputs.append(z)
```



```

states.append(x)

kappa = (rates * self.n).mean(dim=1)
recurrent = kappa[:, None] * self.m[None, :]
external = u[:, t, None] * self.I[None, :]

x = x + self.alpha * (-x + recurrent + external)

outputs = torch.stack(outputs, dim=1)
states = torch.stack(states, dim=1)
return outputs, states

```

Important shape details:

- `kappa` has shape `(batch,)`.
- `kappa[:, None]` has shape `(batch, 1)`.
- `self.m[None, :]` has shape `(1, N)`.
- Their product broadcasts to `(batch, N)`.

16 Training objective

Training means choosing $\mathbf{m}$ and $\mathbf{n}$ so the output sequence z_t matches a target sequence y_t . For the assignment, use a network of size

$$N = 128.$$

The target is only defined during the final decision epoch. If the simulated sequence has length T_{sim} , define the decision window as the last

$$T_{\text{dec}} = 15$$

time steps. The loss should not penalize the network during earlier stimulus or delay time steps. It only asks whether the final decision-period output is close to the target.

For a mini-batch of B trials, the assignment's loss is

$$\mathcal{L}(\mathbf{m}, \mathbf{n}) = \frac{1}{BT_{\text{dec}}} \sum_{b=1}^B \sum_{s=1}^{T_{\text{dec}}} \left(z_{t_s}^{(b)} - y_{t_s}^{(b)} \right)^2, \quad (35)$$

where $t_s = T_{\text{sim}} - T_{\text{dec}} + s$ indexes the final 15 time steps. The problem statement writes the trial index as i ; here b is used instead to avoid confusing trials with neurons.

The instruction says to use stochastic gradient descent, specifically Adam, with

$$B = 32, \quad \eta = 5 \times 10^{-3}.$$

In practice, this means each parameter update uses one mini-batch of 32 trials, computes the average decision-window MSE, backpropagates that scalar loss, and lets Adam update only $\mathbf{m}$ and $\mathbf{n}$. A successful implementation should reach a loss below roughly 5×10^{-2} , possibly after about 1000 parameter updates.

In code:

```

decision_steps = 15

outputs, states = model(u_batch)          # outputs: (batch, time)
output_decision = outputs[:, -decision_steps:]

# If target is already a time series, keep only its decision window.
# If target is one scalar per trial, repeat it across the decision window.
if target_batch.ndim == 2:
    target_decision = target_batch[:, -decision_steps:]
else:
    target_decision = target_batch[:, None].expand_as(output_decision)

loss = ((output_decision - target_decision) ** 2).mean()

```

17 Task-specific training loop

A full training step has four conceptual stages:

1. Generate or sample a mini-batch of input trials and targets.
2. Run the RNN forward through the full input sequence, storing outputs and hidden states.
3. Compute MSE only on the last 15 decision time steps.
4. Backpropagate through the unrolled recurrence and update **m**, **n** with Adam.

```

model = RankOneRNN(n_units=128)
optimizer = torch.optim.Adam(model.parameters(), lr=5e-3)

batch_size = 32
decision_steps = 15

for update in range(1000):
    u_batch, target_batch = make_batch(batch_size)

    optimizer.zero_grad(set_to_none=True)
    outputs, states = model(u_batch)

    output_decision = outputs[:, -decision_steps:]
    if target_batch.ndim == 2:
        target_decision = target_batch[:, -decision_steps:]
    else:
        target_decision = target_batch[:, None].expand_as(output_decision)

    loss = ((output_decision - target_decision) ** 2).mean()
    loss.backward()
    optimizer.step()

    if update % 100 == 0:
        print(update, float(loss))

```

This code assumes `make_batch(32)` returns an input tensor with shape `(32, time)`. The target can either be a full target time series with shape `(32, time)` or one desired decision value per trial with shape `(32,)`.

18 How training works: backpropagation through time

The forward pass unrolls the recurrence:

$$\mathbf{x}_1 = F_\theta(\mathbf{x}_0, u_0), \quad (36)$$

$$\mathbf{x}_2 = F_\theta(\mathbf{x}_1, u_1), \quad (37)$$

$$\vdots \quad (38)$$

$$\mathbf{x}_T = F_\theta(\mathbf{x}_{T-1}, u_{T-1}), \quad (39)$$

where $\theta = \{\mathbf{m}, \mathbf{n}\}$ in this task.

The loss depends on all states through the outputs:

$$\mathcal{L} = \sum_{t \in \text{decision window}} \ell(G(\mathbf{x}_t), y_t).$$

This is important: the error is measured only at the end, but the hidden state at the end depends on every earlier input and every earlier recurrent update. Backpropagation through time is ordinary backpropagation applied to the unrolled computation graph. The same parameters $\mathbf{m}$ and $\mathbf{n}$ are reused at every time step, so their gradients accumulate contributions from every step:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t \in \text{decision window}} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} \frac{\partial \mathbf{x}_t}{\partial \theta}.$$

The term $\partial \mathbf{x}_t / \partial \theta$ already contains the whole history from the initial state up to time t . The dependence of later states on earlier states introduces products of Jacobians:

$$\frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_s} = \prod_{q=s}^{t-1} \frac{\partial F_\theta(\mathbf{x}_q, u_q)}{\partial \mathbf{x}_q}, \quad s < t.$$

If these products shrink, gradients vanish. If they grow, gradients explode. The leak, the factor $\alpha = 0.2$, the low rank, and the saturation of tanh all affect this gradient flow.

PyTorch builds the graph automatically while the loop runs. Calling `loss.backward()` computes the gradients; calling `optimizer.step()` updates only `nn.Parameter` tensors. Do not detach `x` or `states` inside the forward loop during normal training; detaching would cut the gradient path from the decision loss back to earlier stimulus processing. Gradient clipping is not part of the mathematical model, but it can be useful if training produces very large gradients.

19 Debugging failed convergence

If the loss does not get below roughly 5×10^{-2} , first debug the training setup before interpreting the result as a scientific failure.

1. **Overfit one fixed mini-batch.** Generate one batch of 32 trials, reuse exactly that batch for every update, and check whether the loss can be driven very low. If this fails, suspect a bug in the recurrence, target alignment, loss window, or optimizer setup.
2. **Train a full-rank network.** Temporarily replace the rank-one recurrent term by a trainable full matrix. If the full-rank model learns but the rank-one model does not, the data and loss pipeline are probably working. If the full-rank model also fails, the issue is likely in the implementation rather than in the rank constraint.
3. **Inspect dynamics after convergence.** Plot representative inputs, outputs, targets, and hidden-state summaries. Useful hidden summaries include κ_t , the readout z_t , and a few hidden units or low-dimensional projections of **states**.

The most common target-alignment bug is accidentally computing loss over the whole sequence. That trains the network to match undefined or irrelevant targets during stimulus presentation. For this assignment, slice the last 15 time steps:

```
output_decision = outputs[:, -15:]
target_decision = target[:, -15:] # or label[:, None].expand_as(output_decision)
loss = ((output_decision - target_decision) ** 2).mean()
```

20 What gradients hit in this rank-one model

The trainable vectors influence the network differently.

The vector **n** determines what population activity is measured:

$$\kappa_t = \frac{1}{N} \sum_i n_i r_{t,i}.$$

Changing **n** changes the scalar feedback signal extracted from the rates.

The vector **m** determines where that scalar is written back:

$$\text{recurrent input}_{t,i} = m_i \kappa_t.$$

Changing **m** changes which neurons receive positive or negative recurrent drive.

Because **w** is fixed, training cannot directly rotate the output readout. Instead, it must shape the recurrent dynamics so that the fixed readout

$$z_t = \frac{1}{N} \mathbf{w}^\top \tanh(\mathbf{x}_t)$$

becomes task-correct.

21 Low-rank interpretation

A full recurrent matrix has N^2 entries and can mix neural activity in many directions. The rank-one matrix

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top$$

has only two length- N vectors. It performs two operations:

1. read the current rates along $\mathbf{n}$ to get one scalar κ ;
2. write that scalar back into the population along $\mathbf{m}$.

This makes the recurrent part of the dynamics low-dimensional:

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \mathbf{m}\kappa(t) + \mathbf{I}u(t).$$

Even though the state $\mathbf{x}$ has N components, the recurrent feedback has only one degree of freedom. The main computation can often be understood by tracking $\kappa(t)$ and the output $z(t)$.

For intuition, suppose $\tanh(x) \approx x$. Then

$$\tau \dot{\kappa} \approx -\kappa + \sigma_{nm}\kappa + \sigma_{nI}u(t),$$

where

$$\sigma_{nm} = \frac{1}{N} \sum_i n_i m_i, \quad \sigma_{nI} = \frac{1}{N} \sum_i n_i I_i.$$

The overlap σ_{nI} controls how strongly the input drives the latent coordinate. The overlap σ_{nm} controls recurrent feedback. Training $\mathbf{m}$ and $\mathbf{n}$ changes these overlaps and thereby changes whether activity decays quickly, integrates evidence, or moves toward stable decision-like states.

22 Practical implementation checks

Before training on the full task, test the model in small pieces:

1. Check that `model(u)` returns `outputs` with shape `(batch, time)` and `states` with shape `(batch, time, N)`.
2. Check that only `m` and `n` appear in `list(model.parameters())`.
3. Check that `I` and `w` appear in `model.state_dict()` but not in the optimizer parameters.
4. Feed zero input and verify that the zero initial state stays zero.
5. Feed a simple constant input and verify that states and outputs are finite, not `nan` or `inf`.
6. Run one training step and verify that `m.grad` and `n.grad` are not `None`.

Example checks:

```
u = torch.randn(8, 75)
model = RankOneRNN(n_units=128)
outputs, states = model(u)

assert outputs.shape == (8, 75)
assert states.shape == (8, 75, 128)
assert torch.isfinite(outputs).all()

loss = outputs.pow(2).mean()
loss.backward()
assert model.m.grad is not None
assert model.n.grad is not None
assert model.I.grad is None
assert model.w.grad is None
```

23 Common pitfalls

- Forgetting the factor $1/N$. In code, use `mean(dim=1)` or divide sums by `n_units`.
- Accidentally making `I` or `w` trainable. They should be buffers for this task.
- Building the full J matrix unnecessarily. For rank one, use `kappa * m`.
- Mixing time and batch dimensions. Pick one layout and keep it consistent.
- Detaching the state inside the time loop during normal full-sequence training. Detaching cuts the gradient path through time.
- Using in-place tensor operations on values needed for autograd. Prefer assignments like `x = x + ...`
- Computing the output only after the update when the target expects output before the update, or vice versa. Choose the convention deliberately and keep targets aligned with it.

24 Minimal mental model

At each time step, the network does the following:

1. Convert activations to firing rates:

$$\mathbf{r}_t = \tanh(\mathbf{x}_t).$$

2. Read the population along `n`:

$$\kappa_t = \frac{1}{N} \mathbf{n}^\top \mathbf{r}_t.$$

3. Produce the output through the fixed readout:

$$z_t = \frac{1}{N} \mathbf{w}^\top \mathbf{r}_t.$$

4. Combine leak, recurrent feedback, and input:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + 0.2[-\mathbf{x}_t + \mathbf{m}\kappa_t + \mathbf{I}u_t].$$

5. During training, compare z_t to the target, backpropagate through the unrolled steps, and update only `m` and `n`.

That is the core of the assignment: implement this recurrence correctly, generate outputs over time, define a task loss, and let PyTorch’s autograd tune the two low-rank connectivity vectors.

25 Connectivity space: what it is and how the paper analyzes it

The paper uses two complementary views of the same low-rank network.

The first view is *activity state space*. This is the familiar N -dimensional space whose coordinates are the neural activities $x_1, \dots, x_N$. A trajectory in this space is the population activity evolving over time.

The second view is *connectivity space*, also called *loading space* in the paper. This is not the space of activities. It is the space of neuron-specific parameters. Each neuron is represented by one point whose coordinates are that neuron’s entries in the input, recurrent, and readout patterns.

For a network with N_{in} input patterns, one readout pattern, and rank R recurrent connectivity, neuron i has coordinates

$$\mathbf{a}_i = \left(I_i^{(1)}, \dots, I_i^{(N_{\text{in}})}, n_i^{(1)}, m_i^{(1)}, \dots, n_i^{(R)}, m_i^{(R)}, w_i \right). \quad (40)$$

So the loading-space dimension is

$$N_{\text{in}} + 2R + 1$$

when there is one readout. In the assignment’s rank-one, one-input model, each neuron is just a point in a four-dimensional space:

$$\mathbf{a}_i = (I_i, n_i, m_i, w_i). \quad (41)$$

This representation is useful because a trained low-rank RNN is not only a list of vectors $\mathbf{I}, \mathbf{n}, \mathbf{m}, \mathbf{w}$. It is also a cloud of N points $\mathbf{a}_1, \dots, \mathbf{a}_N$ in loading space. The shape of that cloud describes how input drive, recurrent selection, recurrent feedback, and output readout are statistically arranged across neurons.

25.1 The same vectors have two different interpretations

The columns and rows of the connectivity-pattern matrix answer different questions.

As *columns*, the patterns $\mathbf{I}, \mathbf{m}, \mathbf{n}$, and $\mathbf{w}$ are directions across the neural population. For example, $\mathbf{m}$ is a direction in activity state space: when the latent variable κ is positive, recurrent feedback pushes the state along $\mathbf{m}$.

As *rows*, the same numbers define one coordinate vector per neuron. For rank one,

$$\begin{array}{cc} \text{neuron 1} & \left[\begin{array}{cccc} I_1 & n_1 & m_1 & w_1 \end{array} \right] \\ \text{neuron 2} & \left[\begin{array}{cccc} I_2 & n_2 & m_2 & w_2 \end{array} \right] \\ \vdots & \left[\begin{array}{cccc} \vdots & \vdots & \vdots & \vdots \end{array} \right] \\ \text{neuron } N & \left[\begin{array}{cccc} I_N & n_N & m_N & w_N \end{array} \right] \end{array}.$$

Each row is one point in connectivity space. Each column is one population pattern.

This is the central duality in the figures from the paper:

- activity state space explains *where the neural trajectory can go*;
- connectivity space explains *why the reduced trajectory has the particular inputs, feedback loops, and readout it has*.

25.2 From a point cloud to covariances

The paper’s main simplification is that, for large networks, the exact identity of every neuron is less important than the distribution of points in loading space. A common assumption is that the points are sampled from a Gaussian distribution:

$$\mathbf{a}_i \sim \mathcal{N}(0, \Sigma). \quad (42)$$

The covariance matrix Σ summarizes the shape and orientation of the point cloud. Its entries are pattern overlaps:

$$\sigma_{ab} = \frac{1}{N} \sum_{i=1}^N a_i b_i, \quad (43)$$

where a and b can be I , n , m , w , or one of their indexed versions. For example,

$$\sigma_{nI} = \frac{1}{N} \sum_i n_i I_i, \quad \sigma_{nm} = \frac{1}{N} \sum_i n_i m_i, \quad \sigma_{mw} = \frac{1}{N} \sum_i m_i w_i.$$

These covariances are exactly the quantities that appeared earlier in the rank-one latent dynamics. The connectivity-space view explains why: a covariance is a geometric feature of the loading cloud, and the reduced dynamics are controlled by those geometric features.

Pairwise scatterplots are two-dimensional projections of this loading-space cloud. If the scatterplot of n_i against I_i has a strong positive tilt, then neurons with positive input weights also tend to have positive n loadings. The input is therefore read efficiently into κ . If the scatterplot of n_i against m_i has a positive tilt, then the activity written out along $\mathbf{m}$ is read back in by $\mathbf{n}$, creating positive recurrent feedback on κ .

25.3 Why covariances become an effective circuit

For rank one with one input,

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \mathbf{m} \hat{\kappa} + \mathbf{I} u(t), \quad \hat{\kappa} = \frac{1}{N} \mathbf{n}^\top \phi(\mathbf{x}).$$

Because $\mathbf{x}$ is driven only along $\mathbf{m}$ and $\mathbf{I}$, the activity can be described by low-dimensional coordinates. To derive their dynamics carefully, distinguish the rate projection $\hat{\kappa}(t)$ from the coefficient $\kappa(t)$ of the state along $\mathbf{m}$. Also let $v(t)$ be the filtered input, $\tau \dot{v} = -v + u$. Then a state in the low-dimensional subspace has the form

$$x_i(t) = m_i \kappa(t) + I_i v(t). \quad (44)$$

This form is exact when the initial condition lies in the span of $\mathbf{m}$ and $\mathbf{I}$; components outside that span receive no recurrent or external drive and decay through the leak.

Differentiate this expression and substitute it into the full neural equation:

$$m_i \tau \dot{\kappa} + I_i \tau \dot{v} = -m_i \kappa - I_i v + m_i \hat{\kappa} + I_i u.$$

Using $\tau \dot{v} = -v + u$ and matching the coefficient of m_i gives $\tau \dot{\kappa} = -\kappa + \hat{\kappa}$. Inserting the rate projection then closes the dynamics:

$$\tau \dot{\kappa} = -\kappa + \frac{1}{N} \sum_i n_i \phi(m_i \kappa + I_i v(t)). \quad (45)$$

Thus the term $-\kappa$ is inherited from the single-neuron leak, while the summation is the recurrent rate pattern read out by $\mathbf{n}$. The state coordinate κ and the rate projection $\hat{\kappa}$ are equal at a fixed point, but need not be equal during a transient. They are often both denoted by κ when that distinction is not important. Below, we also return to the common shorthand of writing u for the filtered input coordinate v ; for a constant or slowly varying input, v simply follows u .

Mean-field theory is the large-population step that turns this empirical average over a particular set of neurons into an average over a statistical population. If the loading vectors $\mathbf{a}_i$ are independent samples from $p(\mathbf{a})$, then the law of large numbers gives

$$\frac{1}{N} \sum_i f(\mathbf{a}_i) \longrightarrow \int f(\mathbf{a}) p(\mathbf{a}) d\mathbf{a} = \mathbb{E}_p[f(\mathbf{a})]$$

as N becomes large. Here $f(\mathbf{a}) = n\phi(m\kappa + Iu)$, so the finite-network equation becomes

$$\tau \dot{\kappa} = -\kappa + \mathbb{E}_{(I,n,m,w) \sim p(\mathbf{a})} [n\phi(m\kappa + Iu)]. \quad (46)$$

This replacement does not assume that all neurons have the mean loading or the mean activity. It keeps their heterogeneity through the joint distribution of I, n, m , and w ; it only removes the dependence on the identity of each finite sample. The approximation becomes self-averaging: two sufficiently large networks drawn from the same $p(\mathbf{a})$ have nearly the same latent dynamics.

For a zero-mean Gaussian loading distribution, $p(\mathbf{a})$ is completely specified by its covariance matrix. To see how its entries enter, set $h = m\kappa + Iu$. Gaussian integration by parts gives

$$\mathbb{E}[n\phi(h)] = (\sigma_{nm}\kappa + \sigma_{nI}u) \underbrace{\mathbb{E}[\phi'(h)]}_{g(\kappa, u)}, \quad (47)$$

where $g(\kappa, u)$ is the average neural gain. Thus the covariances determine the available input and feedback pathways, while the nonlinearity changes their instantaneous strength through g . For tanh, this gain decreases as more neurons saturate. If the loading distribution is not Gaussian, higher-order features of $p(\mathbf{a})$ can matter as well, so covariances alone need not determine the dynamics.

If activity is close to the linear regime of tanh, then $g(\kappa, u) \approx 1$ (equivalently, $\phi(m\kappa + Iu) \approx m\kappa + Iu$), and

$$\tau \dot{\kappa} \approx -\kappa + \sigma_{nm}\kappa + \sigma_{nI}u. \quad (48)$$

The readout has the analogous approximation

$$z(t) = \frac{1}{N} \sum_i w_i \phi(x_i) \approx \sigma_{wm}\kappa + \sigma_{wI}u. \quad (49)$$

So, in the simplest rank-one case:

- σ_{nI} controls how strongly the input enters the latent variable;
- σ_{nm} controls recurrent feedback and therefore the integration timescale;
- σ_{wm} controls how strongly the latent variable is visible in the output.

For rank R , the same idea becomes a small recurrent circuit with variables $\kappa_1, \dots, \kappa_R$:

$$\tau \dot{\kappa}_r = -\kappa_r + \mathbb{E} \left[n^{(r)} \phi \left(\sum_{q=1}^R m^{(q)} \kappa_q + \sum_s I^{(s)} u_s \right) \right]. \quad (50)$$

In the linear regime, the effective coupling from κ_q to κ_r is set by $\sigma_{n^{(r)}m^{(q)}}$, and the drive from input u_s into κ_r is set by $\sigma_{n^{(r)}I^{(s)}}$. This is why the paper draws the reduced system as an effective circuit: each collective variable is like a unit, and the covariances in connectivity space become the couplings between those units.

25.4 Mean-field theory and the equivalent-circuit task

The previous subsection gives the compact mean-field equation. This subsection unpacks what that approximation does and how to use it for the assignment. For clarity, we keep the filtered input coordinate v explicit rather than replacing it by u .

Step 1: start from the finite trained network. For a particular set of trained loading vectors, the rank-one latent dynamics are

$$\tau \dot{\kappa} = -\kappa + \frac{1}{N} \sum_{i=1}^N n_i \phi(h_i), \quad h_i = m_i \kappa + I_i v, \quad (51)$$

coupled to $\tau \dot{v} = -v + u$. The quantity h_i is the total low-dimensional input current to neuron i . Neurons differ because their coordinates m_i and I_i differ, even though all of them share the same macroscopic variables κ and v .

Mean-field theory does *not* replace every neuron by an average neuron. It replaces the average of many heterogeneous neurons by an expectation over the distribution from which their loadings are drawn:

$$\frac{1}{N} \sum_i n_i \phi(h_i) \longrightarrow \mathbb{E}[n \phi(h)].$$

The individual variation remains inside this expectation. What disappears is the dependence on the identity of a particular finite sample of neurons.

Step 2: find the distribution of neural inputs. Assume the loading coordinates (m, n, I, w) are jointly zero-mean Gaussian. For fixed κ and v ,

$$h = m\kappa + Iv$$

is then Gaussian as well. Its variance is

$$\Delta^2(\kappa, v) = \sigma_m^2 \kappa^2 + \sigma_I^2 v^2 + 2\sigma_{mI} \kappa v. \quad (52)$$

Here $\sigma_m^2 = \mathbb{E}[m^2]$, $\sigma_I^2 = \mathbb{E}[I^2]$, and $\sigma_{mI} = \mathbb{E}[mI]$. If $\mathbf{m}$ and $\mathbf{I}$ are orthogonal in the large-population sense, then $\sigma_{mI} = 0$ and

$$\Delta = \sqrt{\sigma_m^2 \kappa^2 + \sigma_I^2 v^2}. \quad (53)$$

Thus h can be represented as $h = \Delta z$ with $z \sim \mathcal{N}(0, 1)$. Intuitively, Δ measures how widely the neurons are spread across the nonlinear activation function at the current state.

Step 3: average the neuronal gain. The local slope $\phi'(h)$ measures how responsive a neuron is to a small change in its input. Its population average is

$$g(\Delta) = \langle \phi' \rangle(\Delta) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \phi'(\Delta z) e^{-z^2/2} dz. \quad (54)$$

For $\phi = \tanh$, $\phi'(h) = \text{sech}^2(h)$. When Δ is small, most neurons are near the high-gain center of $\tanh$ and $g(\Delta) \approx 1$. When Δ is large, more neurons are saturated and $g(\Delta)$ decreases. This single scalar is how the mean-field circuit retains the effect of the nonlinearity.

Step 4: turn loading covariances into effective couplings. Because n and h are jointly Gaussian, Gaussian integration by parts gives

$$\begin{aligned}\mathbb{E}[n\phi(h)] &= \text{Cov}(n, h) \mathbb{E}[\phi'(h)] \\ &= (\sigma_{nm}\kappa + \sigma_{nI}v) g(\Delta).\end{aligned}\tag{55}$$

The two terms have a direct circuit interpretation. The input is written along $\mathbf{I}$ and read by $\mathbf{n}$, so σ_{nI} sets the input-to-latent pathway. The latent state is written along $\mathbf{m}$ and read back by $\mathbf{n}$, so σ_{nm} sets the recurrent feedback loop. The nonlinearity multiplies both pathways by the current population gain.

Define the state-dependent effective couplings

$$\tilde{\sigma}_{ab}(\kappa, v) = \sigma_{ab} g(\Delta(\kappa, v)).\tag{56}$$

The equivalent circuit is then

$$\begin{aligned}\tau\dot{v} &= -v + u, \\ \tau\dot{\kappa} &= -\kappa + g(\Delta)(\sigma_{nm}\kappa + \sigma_{nI}v).\end{aligned}$$

(57)

It is called one-dimensional because there is one *recurrent* internal variable, κ . Formally, the augmented system also contains v , but v is only an input filter: it is determined by u and receives no feedback from κ .

The readout follows from the same calculation:

$$z_{\text{MF}} = \mathbb{E}[w\phi(h)] = g(\Delta)(\sigma_{wm}\kappa + \sigma_{wI}v).\tag{58}$$

Thus σ_{wm} makes the latent decision variable visible to the output, while σ_{wI} describes any direct input-to-output pathway.

What the assignment is asking you to test. The goal is not merely to integrate another differential equation. It is to test whether a handful of loading-space statistics preserve the computation learned by the full network. A useful sequence is:

1. **Extract the trained statistics.** Inspect the empirical loading means and compute σ_{nm} , σ_{nI} , σ_{wm} , σ_{wI} , σ_m^2 , σ_I^2 , and σ_{mI} . The formulas above assume zero means. Centering is appropriate when testing that model; if the trained means are appreciable, record them because a mean-field model that discards them is testing a different loading distribution.
2. **Check the finite-sum reduction first.** Simulate Eq. (51) with the actual trained loadings. If this does not track the full RNN, first check the coordinate normalization, initial condition, integration step, and the assumption that activity is confined to the span of $\mathbf{m}$ and $\mathbf{I}$.
3. **Simulate the Gaussian mean-field circuit.** For the same input $u(t)$ and time step as the full RNN, update v and κ using Eq. (57). At every step, recompute Δ and evaluate $g(\Delta)$ by numerical quadrature, a precomputed lookup table, or Gaussian samples.
4. **Reconstruct behavior.** Use Eq. (58) to obtain the mean-field output. Apply the same decision rule used for the full network, such as the sign of the final output, and compare accuracy, psychometric curves, latent trajectories, and output trajectories.

5. **Diagnose any mismatch.** If the finite-sum model succeeds but the Gaussian mean-field model fails, the one-Gaussian loading description is probably missing structure, or finite-size effects are important. If both reduced models fail, the issue is more likely the coordinate reduction or its normalization. The paper’s supplied covariance values are a useful reference circuit, but reproducing them is not a substitute for diagnosing the trained network. Before treating supplied numbers as entries of a Gaussian covariance matrix, match the paper’s scaling convention and verify that the matrix is positive semidefinite; in particular, $|\sigma_{ab}| \leq \sigma_a \sigma_b$ must hold.

For intuition, the full network and the mean-field circuit sit at three levels:

$$\begin{aligned}
& \text{trained neurons } (I_i, n_i, m_i, w_i) \\
& \Downarrow \text{ average over the loading cloud} \\
& \text{covariances } \sigma_{nI}, \sigma_{nm}, \sigma_{wm}, \dots \\
& \Downarrow \text{ multiply by the current gain } g(\Delta) \\
& \text{effective input, feedback, and readout couplings.}
\end{aligned}$$

Low rank says that only one recurrent coordinate must be followed. The Gaussian mean-field assumption says that the cloud of individual neurons can be replaced by its covariance geometry. The gain $g(\Delta)$ then tells us how much of that geometric coupling is currently usable rather than lost to saturation.

Normalization convention. Throughout this note, κ is the coefficient in $\mathbf{x} = \mathbf{m}\kappa + \mathbf{I}v$. If $\mathbf{m} \perp \mathbf{I}$, this coefficient is

$$\kappa = \frac{\mathbf{m}^\top \mathbf{x}}{\|\mathbf{m}\|^2}. \quad (59)$$

Some presentations instead use $\mathbf{m}^\top \mathbf{x} / \|\mathbf{m}\|$, the projection length along the unit vector $\mathbf{m} / \|\mathbf{m}\|$. Both conventions are valid, but their numerical κ values differ by a factor of $\|\mathbf{m}\|$, so the state decomposition and covariance factors must be rescaled consistently. Likewise, Eq. (53) contains v , not the raw input u , unless one explicitly makes the slow-input approximation $v \approx u$.

25.5 Single population versus multiple populations

A single Gaussian cloud means all neurons are samples from one statistical population:

$$p(\mathbf{a}) = \mathcal{N}(0, \Sigma).$$

This can already implement many computations. Increasing the rank R increases the number of internal variables κ_r , so the network has more internal degrees of freedom.

The paper then asks a separate question: when is one cloud not enough? To answer this, it allows a mixture of P Gaussian populations:

$$p(\mathbf{a}) = \sum_{p=1}^P \alpha_p \mathcal{N}(0, \Sigma_p). \quad (60)$$

Here α_p is the fraction of neurons in population p , and Σ_p is that population’s covariance matrix. The clusters are centered at the origin in the paper’s model, so what distinguishes populations is not

their mean activity or mean loading. What distinguishes them is their covariance structure: different subpopulations can have different relationships between input, recurrent, and readout patterns.

With multiple populations, the mean-field equation becomes

$$\tau \dot{\kappa}_r = -\kappa_r + \sum_{p=1}^P \alpha_p \mathbb{E}_{\mathbf{a} \sim \mathcal{N}(0, \Sigma_p)} \left[n^{(r)} \phi \left(\sum_q m^{(q)} \kappa_q + \sum_s I^{(s)} u_s \right) \right]. \quad (61)$$

Nonlinearity makes this more powerful than just averaging covariance matrices. The gain $\phi'(x_i)$ can be different in different subpopulations, because an input can push one subpopulation into a high-gain regime and another into a low-gain or saturated regime. In a local linearization, the effective covariance has the form

$$\tilde{\sigma}_{ab} = \sum_{p=1}^P \alpha_p \sigma_{ab}^{(p)} \langle \phi' \rangle_p. \quad (62)$$

The same anatomical connectivity can therefore have different effective low-dimensional couplings depending on which subpopulations are active and what their gains are.

This is the paper’s distinction between rank and population structure:

- R controls how many internal collective variables are available;
- P controls how many statistically distinct neuron groups shape the dynamics of those variables.

25.6 The paper’s analysis pipeline

The paper analyzes trained networks with the following logic.

1. **Train low-rank RNNs on a task.** For each behavioral task, the authors search for a minimal recurrent rank R that can solve it.
2. **Build the loading matrix.** Each neuron becomes one row, with columns given by its input, recurrent, and readout loadings. For example, with one input, one readout, and two recurrent modes, the loading space has six coordinates:

$$(I_i, n_i^{(1)}, m_i^{(1)}, n_i^{(2)}, m_i^{(2)}, w_i).$$

3. **Fit clusters in loading space.** The authors fit Gaussian clusters to the cloud of neuron loadings. They progressively increase the number of clusters P .
4. **Estimate each cluster’s covariance structure.** For every fitted population, they compute the covariance matrix between patterns. This matrix is the compact summary of that population’s connectivity geometry.
5. **Resample networks from the fitted distribution.** They generate new networks by sampling neurons from the fitted Gaussian population or mixture of populations. If the resampled networks solve the task as well as the original trained networks, then the fitted connectivity-space statistics have captured the essential computational structure.
6. **Find the minimal number of populations.** The minimal P is the smallest number of clusters whose resampled networks still perform the task.

7. **Use mean-field theory to identify key parameters.** Once the relevant covariances are known, the paper builds reduced low-dimensional models that keep only the important covariance entries. This reveals the mechanism in terms of input drive, recurrent feedback, coupling between latent variables, and readout.

The important methodological point is that clustering is not used merely for visualization. It is tested causally by resampling: if a simplified connectivity-space distribution can regenerate networks with the same behavior, then that distribution is a valid reduced description of the trained network.

25.7 How the task examples fit the framework

The screenshots summarize the paper’s minimal rank and minimal population counts:

Task	Minimal rank	Minimal populations
Perceptual decision making	$R = 1$	$P = 1$
Multi-sensory decision making	$R = 1$	$P = 1$
Parametric working memory	$R = 2$	$P = 1$
Context-dependent decision making	$R = 1$	$P = 2$
Delay-match-to-sample with two items	$R = 2$	$P = 2$

For perceptual decision making, the network has one input, one readout, and one recurrent mode, so the loading space is four-dimensional: (I, n, m, w) . A single Gaussian population is enough. The key covariances have direct interpretations:

- I - n covariance determines how stimulus input drives κ ;
- n - m covariance determines positive feedback and integration;
- m - w covariance determines how κ drives the readout.

This is the cleanest example of the connectivity-space idea: a four-dimensional point cloud, summarized by a few covariances, becomes a one-dimensional integrator.

For parametric working memory, the network needs $R = 2$. The loading space has six dimensions: one input, one readout, and two pairs of recurrent patterns. The paper finds that one population is still enough, but the covariance structure now creates two different internal variables. One mode has strong positive feedback, so it can maintain the first stimulus through a delay. The other mode is more transient and represents stimulus information without the same persistent feedback. The readout combines the two output directions to compute a difference between the two stimulus values.

For context-dependent decision making, increasing rank alone is not the key move. A rank-one network can solve the task, but it needs two populations. The reason is that context-dependent computations require the same inputs and latent variable to be routed differently depending on context. Distinct subpopulations allow context inputs to modulate gains and effective covariances differently in different groups of neurons. This changes the effective circuit without needing many more recurrent dimensions.

For delay-match-to-sample with two items, the paper reports that both extra rank and extra population structure are needed: $R = 2$ and $P = 2$. In this kind of task, the network needs multiple internal variables and population-specific organization to compare items flexibly.

25.8 How to analyze connectivity space in your rank-one model

After training the assignment model, the direct analogue of the paper’s analysis is to form the $N \times 4$ loading matrix

$$A = \begin{bmatrix} I_1 & n_1 & m_1 & w_1 \\ I_2 & n_2 & m_2 & w_2 \\ \vdots & \vdots & \vdots & \vdots \\ I_N & n_N & m_N & w_N \end{bmatrix}.$$

Then inspect the covariance matrix

$$\Sigma = \frac{1}{N} A^\top A$$

after centering columns if their empirical means are not close to zero. The entries to look at first are:

- $\Sigma_{I,n}$: input-to-latent drive;
- $\Sigma_{n,m}$: recurrent feedback on the latent variable;
- $\Sigma_{m,w}$: latent-to-output readout alignment;
- $\Sigma_{I,w}$: direct input visibility in the readout, if present.

A minimal PyTorch diagnostic is:

```
with torch.no_grad():
    A = torch.stack([model.I, model.n, model.m, model.w], dim=1)
    A = A - A.mean(dim=0, keepdim=True)
    Sigma = A.T @ A / A.shape[0]

print(Sigma)
```

The scatterplots that correspond to the paper’s loading-space projections are just pairwise plots of the columns of A : I versus n , n versus m , m versus w , and so on. These plots are not only descriptive. They tell you which statistical relationships training created. If the trained network integrates evidence, you should expect the learned $\mathbf{n}$ and $\mathbf{m}$ to create recurrent feedback, and the learned $\mathbf{n}$ to align with the fixed input direction enough for stimulus information to enter κ .

25.9 The main takeaway

Connectivity space is the paper’s way of turning a large trained network into a small statistical object. Low rank gives the network a small number of latent variables. The loading-space distribution tells us how the neurons arrange input, recurrent, and readout weights so those latent variables perform a task. The covariance matrix of the loading cloud then becomes the parameter matrix of an effective low-dimensional circuit.

Glossary

Term	Meaning
$\mathbf{x}_t$	recurrent state or activation vector at time step t
$\tanh(\mathbf{x}_t)$	firing-rate vector after the nonlinearity
τ	neuronal time constant; here 100 ms
Δt	Euler step size; here 20 ms
α	discretization factor $\Delta t/\tau = 0.2$
J	recurrent connectivity matrix
rank one	one recurrent read-in/write-out direction pair
$\mathbf{n}$	trainable vector that reads rates into κ
$\mathbf{m}$	trainable vector that writes κ back into neurons
κ_t	scalar recurrent latent coordinate
$\mathbf{I}$	fixed input vector
$\mathbf{w}$	fixed output readout vector
z_t	scalar network output
BPTT	backpropagation through time; backprop on the unrolled recurrence